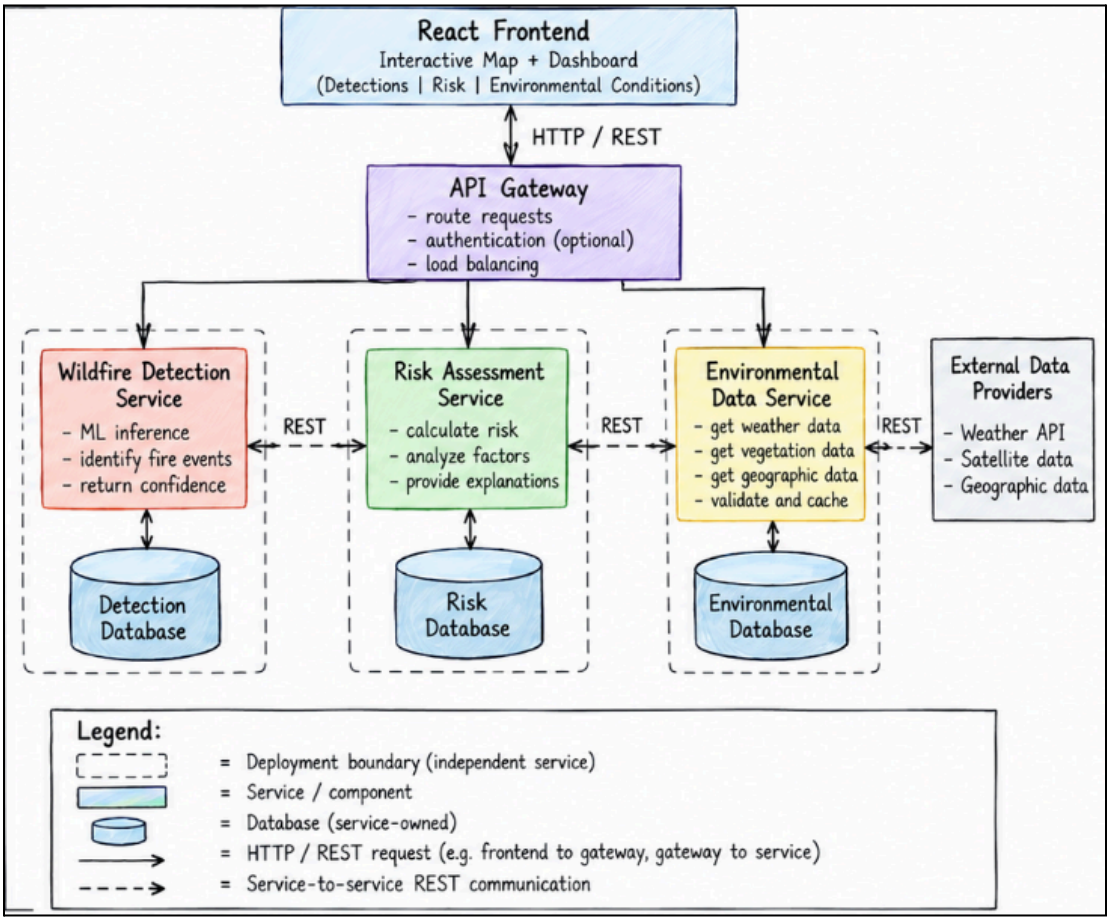
ECE444 PRA5 – Architecture Design
Team Ash Detectives
Wildfire Early Detection and Risk Assessment Platform

1. Introduction

The goal of this project is to develop a platform that detects potential wildfires, assesses wildfire risk, and provides an interactive dashboard for users to visualize results and environmental conditions. This report presents two alternative software architectures for the system: a microservices architecture and a monolithic architecture. Both designs use the same high-level component-and-connector view and include component responsibilities, deployment boundaries, and communication between components.

2. Microservices Architecture

The microservices architecture decomposes the system into three independently deployable backend services: Wildfire Detection, Risk Assessment, and Environmental Data. Each service has a specific responsibility, its own database, and communicates through REST APIs. An API Gateway routes requests from the frontend to the appropriate service.



2.1 Service Responsibilities and Communication

Each microservice provides a distinct function and manages its own data. Services communicate through REST APIs and can be deployed and updated independently.

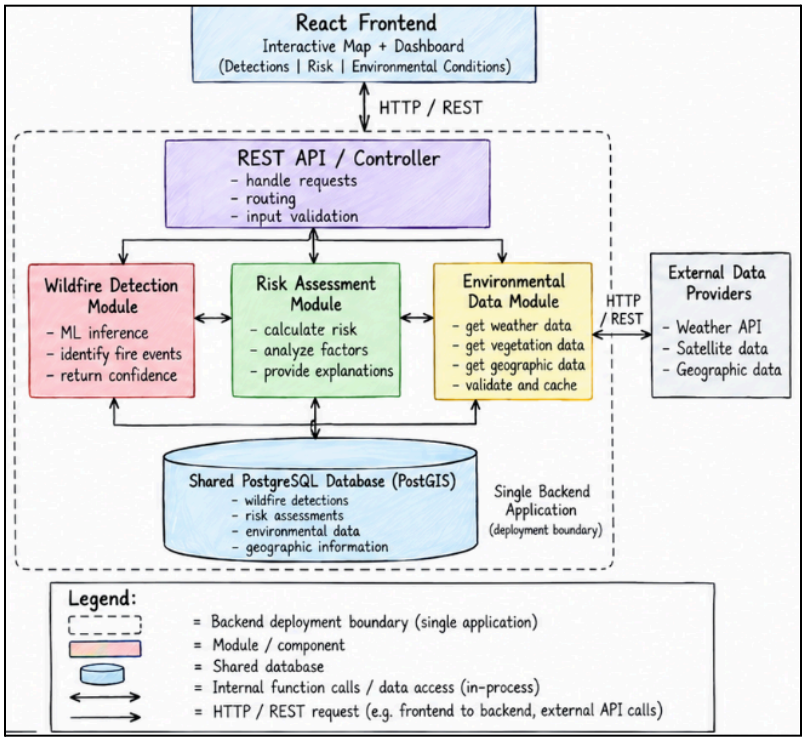
Service	Responsibility	Key API Endpoints
Wildfire Detection Service	Identifies potential wildfire events using machine learning and returns locations and confidence scores.	GET /detections
Risk Assessment Service	Calculates wildfire risk based on environmental factors and provides explanations.	GET /risk
Environmental Data Service	Retrieves and processes weather, vegetation, and geographic data from external providers.	GET /environment

2.2 External Dependencies

The Environmental Data Service communicates with external weather and geospatial data providers. Failures or missing data are handled through timeouts, validation, and cached observations. The Risk Assessment Service and Detection Service use this data through REST requests.

3. Monolithic Architecture

The monolithic architecture combines all backend components into a single deployment unit. The React frontend communicates with one backend application through a REST API. The Wildfire Detection, Risk Assessment, and Environmental Data modules run in the same application process, communicate through internal function calls, and share a single PostgreSQL database.



3.1 Module Responsibilities and Interactions

The monolithic design provides the same functionality as the microservices architecture, but all modules are part of one application.

Module	Responsibility
Wildfire Detection Module	Processes incoming observations, identifies potential wildfire events, and returns locations, timestamps, and confidence scores.
Risk Assessment Module	Calculates wildfire risk using environmental factors and provides explanations of contributing conditions.
Environmental Data Module	Retrieves, validates, and processes environmental and geographic data from external providers.
Shared Database	Stores wildfire detections, geographic information, environmental observations, and risk assessment results.

4. Trade-off Analysis: Reliability and Fault Tolerance

The selected quality requirement is reliability and fault tolerance, based on the project requirement to handle missing data and external service failures, and to communicate unavailable information clearly rather than displaying misleading wildfire assessments.

Criterion	Microservices	Monolithic
Fault isolation	One service can fail while others continue to operate.	A process-level failure may affect the entire backend.
External data failure	Other services may remain operational. Can return cached data if the Environmental Service is down.	Modules can also return cached data, but share the same process.
Deployment reliability	Each service can be deployed and rolled back independently.	All backend modules must be deployed together.
Communication failures	Interservice network calls introduce additional failure points.	Internal calls avoid network failures between modules.
Recovery	Failed services can restart independently.	Entire backend process may need to be restarted.
Operational complexity	Requires service discovery, monitoring, and distributed error handling.	Simpler deployment and centralized monitoring.

4.1 Discussion

Microservices provide better process-level fault isolation. For example, if the Environmental Data Service becomes unavailable, the Detection Service can continue serving previously stored detections, while new detection operations requiring unavailable environmental observations may be delayed or marked as unavailable. The frontend can still display previously detected wildfire events. However, interservice network calls introduce additional failure points and require mechanisms such as timeouts, controlled retries, and error handling. The monolithic architecture avoids network failures between backend modules because they communicate through internal function calls, simplifying error handling and data consistency. If an external weather provider fails, the Environmental Data Module can return cached observations with their timestamps or an explicit unavailable status rather than presenting unsupported risk assessments. However, an unhandled process-level failure or resource exhaustion may interrupt all backend functionality.

5. Recommended Architecture

A modular monolithic architecture is recommended for the current development stage of the Wildfire Early Detection and Risk Assessment Platform. Unlike microservices, the monolithic design uses internal function calls rather than interservice network requests, eliminating network-related failures between backend modules. Its single deployment unit also reduces the operational complexity of coordinating multiple independently deployed services. Reliability can be improved through request timeouts, exception handling, cached environmental observations, and explicit unavailable-data responses. Although the monolith has weaker process-level fault isolation, its simpler architecture is more manageable for the current student team. Clear module boundaries will allow computationally intensive components to be extracted into independent services if future scalability requirements justify the additional complexity.